

CrewSquad COSC 310 Project M1

Project Description:

This project focuses on the design and implementation of a scalable, dockerized backend system for a multi-user food delivery platform. Built using FastAPI, the system models complex domain entities including users, restaurants, orders, and deliveries while strictly adhering to SOLID principles and layered architecture.

Project Goal:

The primary objective is to demonstrate mastery of modern software engineering practices in a real-world context. This includes developing robust RESTful APIs, ensuring data consistency through secure transactions, and establishing a reliable CI/CD pipeline with automated testing. The project prioritizes code maintainability, security, and efficient data retrieval over simple feature implementation.

Function Requirements:

Feature 1:

The system will allow users and restaurant owners/managers to create accounts and log in. It will handle user authentication (login), authorization (what each user is allowed to do), and basic user identity management. Different roles such as regular users and restaurant owners/managers will be supported.

System & Software Requirements:

- The system shall provide an endpoint for users to register with a unique email, password, and role (Customer or Restaurant Owner).
- The system shall authenticate users using JWT (JSON Web Tokens) to maintain secure sessions.
- The system shall enforce Role-Based Access Control (RBAC), ensuring only Restaurant Owners can modify menu data.
- The system shall securely hash passwords before storing them in the database.

User Stories:

- As a new customer, I want to create an account with my email and password so that I can save my delivery address and order history.
- As a returning user, I want to log in securely so that I can access my personal dashboard and shopping cart.

- As a restaurant owner, I want to register my business so that I can manage my restaurant's digital storefront and menu.
- As a user, I want my password to be encrypted so that my personal account remains secure even if the database is compromised.

Feature 2:

The system will store information about restaurants and their menus. It will ensure that data is valid, properly connected (for example, menu items must belong to a restaurant), and that basic constraints are enforced, such as preventing invalid or missing values.

System & Software Requirements:

- The system shall allow Restaurant Owners to add, update, or delete menu items associated with their specific restaurant ID.
- The system shall validate that every menu item contains a name, price, and category (ex. Appetizer, Main, Drink).
- The system shall validate item prices and restaurant cuisine types to ensure data consistency.
- The system shall maintain referential integrity, ensuring no menu items exist without a parent restaurant.

User Stories:

- As a restaurant owner, I want to add new dishes to my menu with prices and descriptions so that customers know what I am currently serving.
- As a customer, I want to view a restaurant's full menu organized by categories so that I can easily find the type of food I'm looking for.
- As a restaurant owner, I want to ensure my menu prices are accurate so that I don't lose money on orders.
- As a customer, I want to see that every dish in the app belongs to an active, valid restaurant, so that I don't order items that cannot be delivered.

Feature 3:

The system will allow users to browse restaurant menus and search for items or restaurants. Backend logic will handle filtering, searching, and returning paginated results

System and Software requirements:

- The system shall provide an API endpoint that allows users to filter restaurants specifically by name, cuisine type, or minimum rating.
- The system shall prevent performance issues by limiting the number of results returned in a single request, for example: 10 at a time. Using limit and offset parameters.

- The system shall provide database queries that must support sorting results efficiently by Price as ascending or Rating as descending before sending them to the user.
- The system shall return a helpful message if the user searches for a type of food and restaurant and there are no results, instead of an error message.

User Stories:

- As a hungry customer, I want to search for restaurants by name, cuisine type, or rating, so that I can quickly find a place that matches my specific wants.
- As a user with a bad internet connection, I want the system to return restaurant menus in small amounts, so that the application loads quickly and doesn't overwhelm me with data.
- As a budget-conscious student, I want to sort search results by Price (Low to High) or Rating (High to Low), so that I can prioritize restaurants that fit my budget or quality standards.
- As a user searching for a specific food, I want to receive a useful message or suggestion if my search yields no results, so that I am not left looking at a blank screen and can find an alternative option.

Feature 4:

The system will allow users to create and manage food orders. It will ensure that orders are consistent, correctly stored, and follow business logic for the domain (for example, an order cannot be modified after it is completed).

System and Software requirements:

- The system shall validate that every item in a new order belongs to the same restaurant, mixed-restaurant orders will be rejected.
- The system shall make sure the backend will enforce a strict system where orders can only move forward (Pending → Preparing → Delivered) and cannot skip steps.
- The system shall make sure that once an order reaches a final status like Delivered or Cancelled, the system must lock the record so no further changes can be made.
- The system shall confirm that the API must ensure that only Restaurant Managers can Accept or Reject orders, while Customers can only Cancel their own orders before preparation starts.

User Stories:

- As a customer, I want to create a new order with multiple items from a single restaurant, so that I can get all my food delivered in one go.
- As a restaurant manager, I want to prevent any modifications to an order once it is marked as Completed or Cancelled, so that our financial records and delivery metrics are accurate and straightforward.

- As a restaurant manager, I want to view a queue of incoming Pending orders and explicitly accept or reject them, so that I can manage my kitchen's capacity effectively.
- As a customer, I want to be able to cancel my order if the restaurant hasn't started preparing it yet, so that I can correct a mistake without causing food waste or financial loss.

Feature 5:

The system will manage delivery-related information. It will support assigning deliveries and tracking basic delivery status as part of the backend logic.

System & Software Requirements:

- The system shall broadcast the order to drivers that are close to the restaurant.
- The system shall validate the location the user enters when setting/updating location.
- The system shall share the location & delivery instructions to the driver.
- The system shall allow users to change their location, but this should not affect ongoing orders.

User Stories:

- As a customer, I want to be able to leave a note on how to drop off the food to drivers (should the driver leave the food somewhere or hand it to the customer, building access code, etc), so the driver won't be confused on arrival.
- As a food delivery driver, I want to be able to filter orders by distance, so I can fulfil my orders more efficiently.
- As a customer, I want to be able to change my delivery location, so I can order food in other cities.
- As a customer, I want to be able to order from the closest available brand restaurant (like KFC, Subway, etc) by default, so I can get my food fresh.

Feature 6:

The system will calculate the total cost of an order, including item prices, delivery fees, and taxes. These calculations will follow predefined business rules implemented in the backend.

System & Software Requirements:

- The system shall compute the taxes based on region, delivery fee based on distance and road/weather condition, and apply other additional fees if applicable (like beverage recycle fee)
- The system shall show all the fees (individual item price, delivery fee, taxes, service fee, beverage fee) to users on checkout.
- The system shall save the receipt in order history to allow the user to view it later.
- The system shall allow restaurant owners to adjust item prices, but this should not affect orders that have already been placed.

User Stories:

- As a customer, I want to be able to view each fee before ordering, so I can know exactly how much I will have to spend on each fee.
- As a customer, I want to be able to view each fee after placing the order, so I can keep a record of my purchase.
- As a restaurant owner, I want to make the adjusted item price update on the user-end immediately, so when I make limited time offers, users can see it without needing to refresh the page.
- As a customer, I want the system to adjust the delivery fee / service fee based on the supply and demand (like raising the delivery fee in bad weather), so orders won't be left unhandled.

Feature 7:

The system will simulate payment processing. No real payment gateway will be used, but the system will follow the correct workflow for accepting or rejecting a payment and updating the order status.

System & Software Requirements

- The system shall create and store a payment attempt whenever a user submits an order, this includes the order's ID, amount to be spent, with a status of the order. The initial status will be PENDING
- The system shall give the user the decision to accept or reject a payment attempt with a box for reason
- The system shall simulate the payment outcome, via whether the payment was accepted or rejected, and the payment result will either be timestamped with an ACCEPTED status or REJECTED status with a reason given
- The system shall update the order status based on the given payment outcome and display the result to the user

User Stories

- As a restaurant owner, I want to choose to accept or reject an order, so that I can ensure my workers aren't overwhelmed
- As a restaurant owner, I want to have the ability to give a reason for rejecting an order, so that I can maintain that customer's loyalty
- As a user, I want my payment to be simulated when I place an order so that I can experience a realistic checkout process.
- As a user, I want to see that my order and payment have been accepted, so that I know if I have to change restaurants

Feature 8:

The system will generate notifications or events when important actions occur, such as order creation or status changes.

System & Software Requirements

- The system shall create and store a notification when an order is made, this includes the notification type, order ID, restaurant ID, and a timestamp
- The system shall create and store a notification when the order status changes and ensure that it reaches the correct recipients i.e. the user and restaurant owner
- The system shall contain an interface that holds a list of notifications for the current order to give the recipient a timeline of events
- The system shall notify the restaurant owner of a new order as soon as possible so they can respond quickly to the order

User Stories

- As a restaurant owner, I want to be notified when a new order is placed, so that I can respond quickly
- As a restaurant owner, I want to see a list of recent orders, so that I can track previous orders and ensure how many must be fulfilled
- As a user, I want to receive updates when my order status changes, so that I can see what's happening with my food
- As a user, I want to receive a notification when my payment goes through, so that I know the order is getting prepared

Non-Functional Requirements:

- The backend API shall respond to 95% of read requests (fetching menus, searching restaurants) within 300 milliseconds under a load of 50 concurrent users.
- Database queries for search and filtering shall not exceed 500 milliseconds for datasets up to 10,000 records.
- All user passwords must be hashed using a strong cryptographic algorithm before storage, plain-text passwords shall never be stored.
- API Access Tokens shall have a strict expiration time of 60 minutes to minimize security risks from session hijacking.
- The codebase must maintain a minimum Unit Test Coverage of 80% as reported by pytest in the CI pipeline.
- The system shall be fully deployable via command "docker-compose up" on any OS (Windows, Mac, Linux) with Docker installed, without requiring manual dependency installation on the host machine.

Domain Requirements:

- An order must strictly contain items from one single restaurant. The system shall forbid mixing items from different restaurants into one order.
- An order's lifecycle is strictly linear. An order cannot revert to a previous state. For example: An order that has been delivered cannot go back to being prepared.
- The price of an item in a completed order must reflect the price at the time of purchase. Subsequent updates to the Restaurant Menu price must not change to the total cost of previous orders that users have paid for.
- All monetary values (prices, fees, taxes) shall be calculated and stored with exactly two decimal places (penny) of precision to prevent floating-point rounding errors. So \$5.99 and not \$5.99000001
- The core Restaurants and Menu data imported from the Kaggle dataset shall remain read-only regarding their primary keys (ID) to ensure data consistency with the external source.
- Every entity in the system (User, Restaurant, Order, Menu Item) must be identifiable by a Unique ID that never changes once it has been assigned.

Contribution:

Branden Skoupas: Contributed to 25% of this Milestone. Came up with the System requirements and user stories for features 1 and 2. Created the traceability matrix.

Keir Parnitzke: Contributed 25% to this Milestone. Came up with the System requirements and user stories for features 3 and 4. Set up the github repo and the kanban board and invited everyone so we could all collaborate on it.

Zijian Chen: Contributed 25% to this Milestone. (User stories and system software requirements for feature 5 & 6.

Paul Daoud: Contributed 25% to this Milestone. (User stories and system software requirements for feature 7 & 8.

Requirements traceability matrix:

CF IDs	Core Features	FR IDs	Functional Requirements	US IDs	User Stories
Feat1	The system will allow users and restaurant owners/managers to create accounts and log in. It will handle user authentication (login), authorization (what each user is allowed to do), and basic user identity management. Different roles such as regular users and restaurant owners/managers will be supported.	Feat1-FR1	The system shall provide an endpoint for users to register with a unique email, password, and role (Customer or Restaurant Owner).	Feat1-US1	As a new customer, I want to create an account with my email and password so that I can save my delivery address and order history.
		Feat1-FR2	The system shall authenticate users using JWT (JSON Web Tokens) to maintain secure sessions.	Feat1-US2	As a returning user, I want to log in securely so that I can access my personal dashboard and shopping cart.
		Feat1-FR3	The system shall enforce Role-Based Access Control (RBAC), ensuring only Restaurant Owners can modify menu data.	Feat1-US3	As a restaurant owner, I want to register my business so that I can manage my restaurant's digital storefront and menu.
		Feat1-FR4	The system shall securely hash passwords before storing them in the database.	Feat1-US4	As a user, I want my password to be encrypted so that my personal account remains secure even if the database is compromised.
Feat2	The system will store information about restaurants and their menus. It will ensure that data is valid, properly connected (for	Feat2-FR1	The system shall allow Restaurant Owners to add, update, or delete menu items associated with	Feat2-US1	As a restaurant owner, I want to add new dishes to my menu with prices and descriptions so that customers know what I am currently serving.

	example, menu items must belong to a restaurant), and that basic constraints are enforced, such as preventing invalid or missing values.		their specific restaurant ID.		
		Feat2 -FR2	The system shall validate that every menu item contains a name, price, and category (ex. Appetizer, Main, Drink).	Feat2 -US2	As a customer, I want to view a restaurant's full menu organized by categories so that I can easily find the type of food I'm looking for.
		Feat2 -FR3	The system shall validate item prices and restaurant cuisine types to ensure data consistency.	Feat2 -US3	As a restaurant owner, I want to ensure my menu prices are accurate so that I don't lose money on orders.
		Feat2 -FR4	The system shall maintain referential integrity, ensuring no menu items exist without a parent restaurant.	Feat2 -US4	As a customer, I want to see that every dish in the app belongs to an active, valid restaurant, so that I don't order items that cannot be delivered.
Feat3	The system will allow users to browse restaurant menus and search for items or restaurants. Backend logic will handle filtering, searching, and returning paginated results	Feat3 -FR1	The system shall provide an API endpoint that allows users to filter restaurants specifically by name, cuisine type, or minimum rating.	Feat3 -US1	As a hungry customer, I want to search for restaurants by name, cuisine type, or rating, so that I can quickly find a place that matches my specific wants.
		Feat3 -FR2	The system shall prevent performance issues by limiting the number of results returned in a single request, for example: 10 at a time. Using limit and offset parameters.	Feat3 -US2	As a user with a bad internet connection, I want the system to return restaurant menus in small amounts, so that the application loads quickly and doesn't overwhelm me with data.

		Feat3 -FR3	The system shall provide database queries that must support sorting results efficiently by Price as ascending or Rating as descending before sending them to the user.	Feat3 -US3	As a budget-conscious student, I want to sort search results by Price (Low to High) or Rating (High to Low), so that I can prioritize restaurants that fit my budget or quality standards.
		Feat3 -FR4	The system shall return a helpful message if the user searches for a type of food and restaurant and there are no results, instead of an error message.	Feat3 -US4	As a user searching for a specific food, I want to receive a useful message or suggestion if my search yields no results, so that I am not left looking at a blank screen and can find an alternative option.
Feat4	The system will allow users to create and manage food orders. It will ensure that orders are consistent, correctly stored, and follow business logic for the domain (for example, an order cannot be modified after it is completed).	Feat4 -FR1	The system shall validate that every item in a new order belongs to the same restaurant, mixed-restaurant orders will be rejected.	Feat4 -US1	As a customer, I want to create a new order with multiple items from a single restaurant, so that I can get all my food delivered in one go.
		Feat4 -FR2	The system shall make sure the backend will enforce a strict system where orders can only move forward (Pending, Preparing, Delivered) and cannot skip steps.	Feat4 -US2	As a restaurant manager, I want to prevent any modifications to an order once it is marked as Completed or Cancelled, so that our financial records and delivery metrics are accurate and straightforward.
		Feat4 -FR3	The system shall make sure that once an order	Feat4 -US3	As a restaurant manager, I want to view a queue of

			reaches a final status like Delivered or Cancelled, the system must lock the record so no further changes can be made.		incoming Pending orders and explicitly accept or reject them, so that I can manage my kitchen's capacity effectively.
		Feat4 -FR4	The system shall confirm that the API must ensure that only Restaurant Managers can Accept or Reject orders, while Customers can only Cancel their own orders before preparation starts.	Feat4 -US4	As a customer, I want to be able to cancel my order if the restaurant hasn't started preparing it yet, so that I can correct a mistake without causing food waste or financial loss.
Feat5	The system will manage delivery-related information. It will support assigning deliveries and tracking basic delivery status as part of the backend logic	Feat5 -FR1	The system shall broadcast the order to drivers that are close to the restaurant.	Feat5 -US1	As a food delivery driver, I want to be able to filter orders by distance, so I can fulfil my orders more efficiently.
		Feat5 -FR2	The system shall validate the location the user enters when setting/updating location.	Feat5 -US2	As a customer, I want to be able to change my delivery location, so I can order food in other cities.
		Feat5 -FR3	The system shall share the location & delivery instructions to the driver.	Feat5 -US3	As a customer, I want to be able to leave a note on how to drop off the food to drivers (should the driver leave the food somewhere or hand it to the customer, building access code, etc), so the driver

					won't be confused on arrival.
		Feat5 -FR4	The system shall allow users to change their location, but this should not affect ongoing orders.	Feat5 -US4	As a customer, I want to be able to order from the closest available brand restaurant (like KFC, Subway, etc) by default, so I can get my food fresh.
Feat6	The system will calculate the total cost of an order, including item prices, delivery fees, and taxes. These calculations will follow predefined business rules implemented in the backend.	Feat6 -FR1	The system shall compute the taxes based on region, delivery fee based on distance and road/weather condition, and apply other additional fees if applicable (like beverage recycle fee).	Feat6 -US1	As a customer, I want to be able to view each fee before ordering, so I can know exactly how much I will have to spend on each fee.
		Feat6 -FR2	The system shall show all the fees (individual item price, delivery fee, taxes, service fee, beverage fee) to users on checkout.	Feat6 -US2	As a customer, I want to be able to view each fee after placing the order, so I can keep a record of my purchase.
		Feat6 -FR3	The system shall save the order history and allow the user to view it later.	Feat6 -US3	As a restaurant owner, I want to make the adjusted item price update on the user-end immediately, so when I make limited time offers, users can see it without needing to refresh the page.

		Feat6 -FR4	The system shall allow restaurant owners to adjust item prices, but this should not affect orders that have already been placed.	Feat6 -US4	As a customer, I want the system to adjust the delivery fee / service fee based on the supply and demand (like raising the delivery fee in bad weather), so orders won't be left unhandled.
Feat7	The system will simulate payment processing. No real payment gateway will be used, but the system will follow the correct workflow for accepting or rejecting a payment and updating the order status.	Feat7 -FR1	The system shall create and store a payment attempt whenever a user submits an order, this includes the order's ID, amount to be spent, with a status of the order. The initial status will be PENDING.	Feat7 -US1	As a restaurant owner, I want to choose to accept or reject an order, so that I can ensure my workers aren't overwhelmed.
		Feat7 -FR2	The system shall give the user the decision to accept or reject a payment attempt with a box for reason.	Feat7 -US2	As a restaurant owner, I want to have the ability to give a reason for rejecting an order, so that I can maintain that customer's loyalty.
		Feat7 -FR3	The system shall simulate the payment outcome, via whether the payment was accepted or rejected, and the payment result will either be timestamped with an ACCEPTED status or REJECTED status with a reason given.	Feat7 -US3	As a user, I want my payment to be simulated when I place an order so that I can experience a realistic checkout process.
		Feat7 -FR4	The system shall update the order	Feat7 -US4	As a user, I want to see that my order and

			status based on the given payment outcome and display the result to the user.		payment have been accepted, so that I know if I have to change restaurants.
Feat8	The system will generate notifications or events when important actions occur, such as order creation or status changes.	Feat8 -FR1	The system shall create and store a notification when an order is made, this includes the notification type, order ID, restaurant ID, and a timestamp.	Feat8 -US1	As a restaurant owner, I want to be notified when a new order is placed, so that I can respond quickly.
		Feat8 -FR2	The system shall create and store a notification when the order status changes and ensure that it reaches the correct recipients i.e. the user and restaurant owner.	Feat8 -US2	As a restaurant owner, I want to see a list of recent orders, so that I can track previous orders and ensure how many must be fulfilled.
		Feat8 -FR3	The system shall contain an interface that holds a list of notifications for the current order to give the recipient a timeline of events.	Feat8 -US3	As a user, I want to receive updates when my order status changes, so that I can see what's happening with my food.
		Feat8 -FR4	The system shall notify the restaurant owner of a new order as soon as possible so they can respond quickly to the order.	Feat8 -US4	As a user, I want to receive a notification when my payment goes through, so that I know the order is getting prepared.

Backlog URL: <https://github.com/users/KeirPar/projects/1>

Repo URL: https://github.com/KeirPar/CrewSquad_310